



Kod Sağlamaştırma: C/C++

CEN429 Güvenli Programlama — Hafta 4

Dr. Öğr. Üyesi Uğur CORUH · 09.10.2026

Bugünün planı (3 saat)

Saat	Bölüm	Konu
1	1	Sağlamlaştırma (hardening) katmanları · SEI CERT · girdi doğrulama ilkeleri · CERT çiftleri
2	2–4	Biçim dizesi (format string) Demo 1 · UAF Demo 2 · tamsayı/UB Demo 3 · hata işleme/sinyaller · statik analiz · sanitizer · fuzzing Demo 4
3	5–7	Derleyici/OS korumaları Demo 5 · güvenli derleme hattı (CI) · kod gizleme Demo 6–7 · uçtan uca · proje

Öğrenme çıktısı: ÖÇ.3 (binary uygulama korumaları)

Kısa tarihçe — bellek hataları ve savunmaların yarışı

- 1988 — **Morris Worm** tampon taşmasını dünyaya duyurur
- 1996 — *Smashing the Stack for Fun and Profit* sömürüyü herkese öğretir
- 1998 → 2004 — **kanarya** (StackGuard) · **ASLR** (PaX) · **DEP/NX** · FORTIFY
- 2012–13 — **AddressSanitizer** ve **fuzzing** (AFL) hataları otomatik yakalar

Ders bu yarış izler: hatayı yaz → araç yakalasın → derleyici/OS korusun.

Önceki haftalardan gelenler

- **Süreç belleği: yığın ve öbek** — yığın (stack) fonksiyon çağrılarının yerel değişkenlerini otomatik yönetir, öbek (heap) `malloc` / `free` ile elle yönetilir (**Hafta 1**)
- **Arabellek taşması** — bir diziye ayrılan boyuttan fazla veri yazmak; taşan baytlar komşu belleğe yazılır (**Hafta 1**)
- **Yığın çerçevesi ve dönüş adresi** — bir fonksiyon çağrısının yerel değişkenlerini ve dönüş adresini tuttuğu bölüm (**Hafta 1**)
- **Beyaz kutu saldırgan modeli** — cihazın sahibi aynı zamanda olası saldırgandır; belleği okuyabilir, hata ayıklayıcı bağlayabilir (**Hafta 1**)
- **CWE** — yazılım zayıflıklarının numaralı kataloğu (**Hafta 2**)

Bu hafta: 1–3. haftanın ilke/tehdit modeli/kripto temeli üzerine SEI CERT kuralları, sanitizeler'lar ve derleyici/OS korumalarını kuruyoruz.

Bu haftanın kavramları

Her terim, gövdede ilk geçtiği yerde tanımlanır; burada yalnız **nerede** olduğunu işaretliyoruz.

Kavram	Nerede
Sağlamlaştırma katmanları · SEI CERT	Bölüm 1
Girdi doğrulama ilkeleri	Bölüm 1
Biçim dizesi açığı	Bölüm 2
Serbest bırakılmış bellek (UAF)	Bölüm 3
Tanımsız davranış (UB)	Bölüm 3
Hata işleme, sinyaller	Bölüm 3
Statik analiz · sanitizer · fuzzing	Bölüm 4
Derleyici/OS korumaları · CI	Bölüm 5
Kod gizleme	Bölüm 6

İşaretçi (pointer)

- **İşaretçi:** bir bellek **adresini** tutan değişken.
- `p` bir adres; `*p` o adresteki değer.
- Yanlış adres → çökme ya da yanlış veri.

Derleyici bayrağı (flag)

- **Bayrak:** derleyiciye verilen seçenek.
- Örnek: `-O2` (optimizasyon), `-Wall` (uyarılar), `-fsanitize=address`.
- Doğru bayraklar birçok hatayı **derleme anında** yakalar.

Uyarı (warning) vs hata (error)

- **Hata:** derleme durur.
- **Uyarı:** derleme sürer ama bir sorun bildirilir.
- Kural: uyarıları **hataya çevir** (`-Werror`) — görmezden gelinen uyarı, gelecekteki açıktır.

Demolar nasıl çalışıyor?

- Demolar `code/week-04` altında; **Windows (Visual Studio 2022)** ve **WSL/Linux**'ta çalışır.
- Derle: Windows `.\build.ps1` · WSL/Linux `./build.sh`
- Çalıştır: her demo klasöründe Windows `.\demo.ps1` · WSL/Linux `sh demo.sh`
- **Fuzzing (Demo 4)** Linux/WSL'de **clang** ister (`sudo apt install -y clang`); Windows'ta VS'nin "C++ AddressSanitizer" bileşeni gerekir.

 **Etik:** demolar yalnız **kendi küçük programlarımızda**; başka bir programa/sisteme dokunmaz.

1. Katmanlar ve SEI CERT

Üç katman, doğru sıra

Kod sağlamlaştırma üç katmandır. Sıra önemli:

1. **Önce hatasız kod** (güvenli kodlama)
2. **Sonra zararı sınırla** (derleyici/OS korumaları)
3. **En son okumayı zorlaştır** (gizleme)



Gizleme, hatayı **düzeltilmez**.

Neden bu sıra?

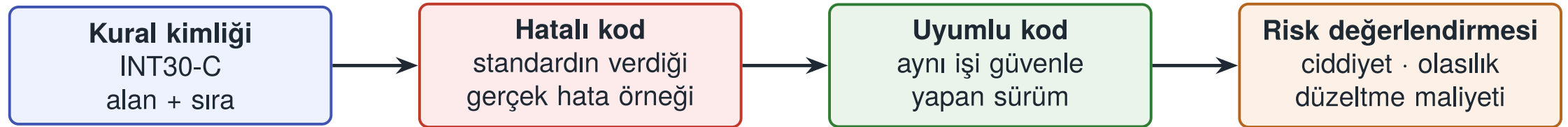
- Hatalı koda koruma eklemek, çürük temele boya çekmek gibidir.
- Önce hatayı **kaldır**; sonra kaçanı **yakala**; en son tersine mühendisliği **yavaşlat**.
- Bu hafta üçünü de göreceğiz, bu sırayla.

CERT kuralının anatomisi — şema

SEI CERT C/C++: gerçek zafiyetlerden çıkarılan, güvenli kodlamanın kural kitabı.

SEI CERT kuralının anatomisi

her kural aynı dört parçayla gelir

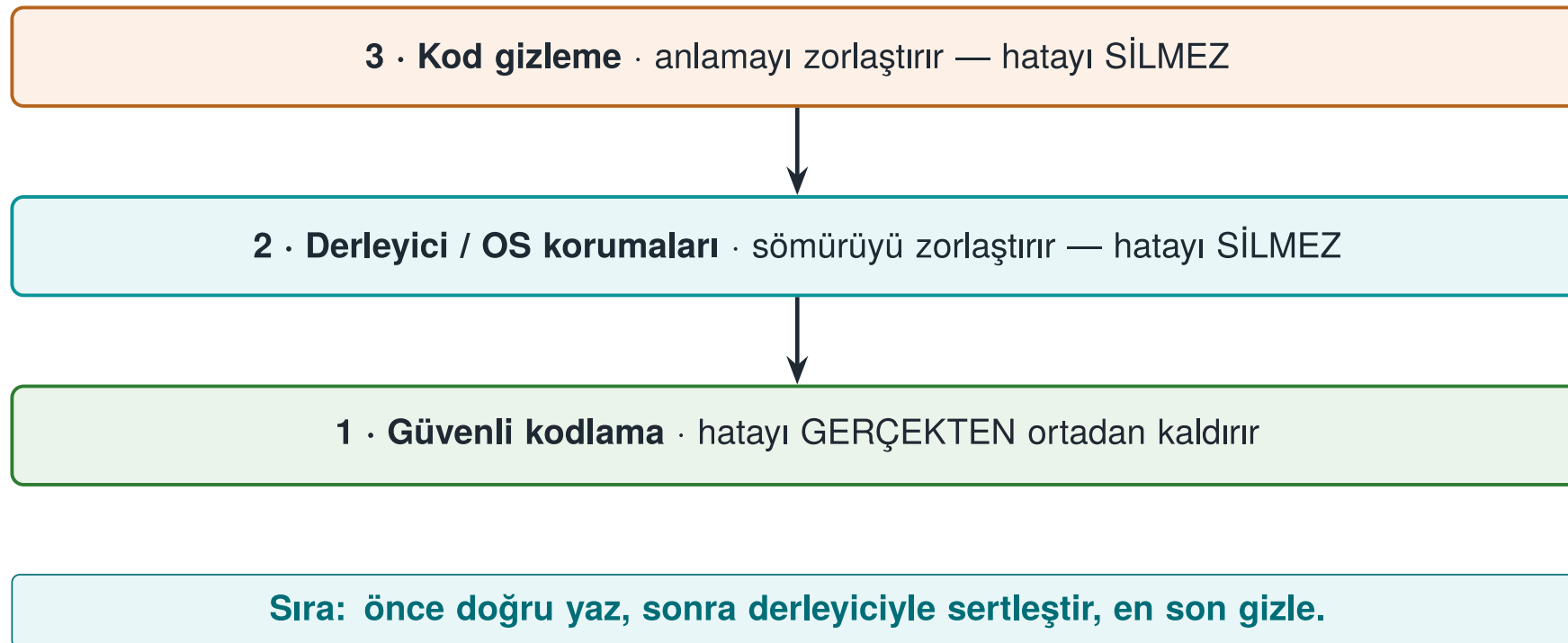


Kuralı ezberlemeyin: hatalı ile uyumlu kodun FARKINI okuyun; öğretici olan orasıdır.

Üç katman — şema

Kod sağlamlaştırmanın üç katmanı

yalnız en alttaki katman hatayı yok eder



Katman 2 · Güvenli kodlama

- **Soru:** kodumda hata var mı?
- **Araçlar:** CERT kuralları, sanitizer, fuzzing.
- **Amaç:** hatayı en baştan **yazmamak**.

Katman 3 · Derleyici/OS

- **Soru:** hata kaçarsa sömürü zor mu?
- **Araçlar:** kanarya, FORTIFY, ASLR, NX, RELRO, CFI.
- **Amaç:** kalan hatanın **istismarını** pahalı kılmak.

Katman 4 · Gizleme

- **Soru:** kodu okuyan anlıyor mu?
- **Araçlar:** sembol/dize gizleme, günlük kaldırma, düzleştirme.
- **Amaç:** tersine mühendisliği **yavaşlatmak**.

Sahadan: şema gereksinimleri

Sertifikalı bir üründe:

- "Yaygın zafiyetler ele alınmalı: hassas veri, enjeksiyon, taşma, dile özgü zayıflıklar"
- "Savunmacı programlama **kodun tamamında tutarlı**"
- "Statik ve dinamik tersine mühendisliğe karşı koruma"

→ Yalnız native tarafta **18 sağlamlaştırma önlemi**.

SEI CERT: bir kural neye benzer?

Her CERT kuralının parçaları:

- **Kimlik:** STR31-C , MEM50-CPP
- **Başlık:** tek cümle
- **Hatalı örnek** ve **uyumlu çözüm**
- **Risk düzeyi** (L1/L2/L3) ve **CWE** bağı

Kural mı, öneri mi?

- **Kural (rule):** ihlali bir **zafiyettir**, denetlenebilir.
- **Öneri (recommendation):** iyi uygulama.
- İncelemeye **L1** (en yüksek risk) kurallarından başla.

Risk düzeyi nasıl belirlenir?

Risk = önem × olasılık × düzeltme maliyeti.

- **L1:** yüksek risk, önce bunlar.
- **L2 / L3:** giderek düşer.

CERT kategorileri (1)

Kısaltma	Kategori	Örnek
INT	Tamsayılar	INT32-C: işaretli taşmaya izin verme
ARR/STR	Dizi/dize (string)	STR31-C: yeterli yer ayır
MEM	Bellek	MEM30-C: bırakılmış belleğe erişme
FIO	Dosya G/Ç	FIO30-C: girdiyi biçim dizesinden dışla

CERT kategorileri (2)

Kısaltma	Kategori	Örnek
ENV	Ortam	ENV33-C: <code>system()</code> çağırma
SIG	Sinyaller	SIG30-C: işleyicide yalnız güvenli fn.
ERR	Hata işleme	ERR33-C: kütüphane hatasını algıla
CON/MSC	Eşzamanlılık/çeşitli	CON43-C, MSC32-C

C++: MEM50-CPP, CTR50-CPP, STR50-CPP, EXP53-CPP, ERR50-CPP.

Otomatik CERT denetimi

- Uyarılar: `-Wall -Wextra -Wformat=2 -Wconversion`
- `clang-tidy cert-*`
- `cppcheck --addon=cert`
- MSVC `/analyze`

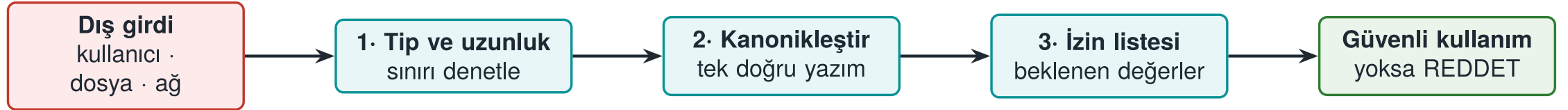
Değerlendirici önce bu taramayı çalıştırır, sonra elle okur.

Girdi doğrulama

Girdi doğrulama — şema

Güven sınırında girdi doğrulama

her dış girdi bu kapıdan geçer



Kara liste değil İZİN listesi: neyin yasak olduğunu değil, neyin serbest olduğunu yazın.

Girdi nereden gelir?

Sandığınızdan çok yerden. Ve **hepsi güvenilmez**:

- Komut satırı, ortam değişkenleri
- Dosya, ağ
- IPC (aynı makinedeki başka süreç)
- Veritabanı, kütüphane arayüzü (JNI)

Sık unutulan girdiler

- `argv[0]` bile saldırganın olabilir; `PATH` , `LANG` .
- Dosya biçimindeki **uzunluk alanları**.
- Ağ: "16 bayt" der, 1 bayt gönderir.
- "Kendi verimiz" sonradan değişmiş olabilir.

Girdi doğrulamanın beş ilkesi

1. **Beyaz liste** (izin verilenleri say), kara liste değil.
2. Önce **kanonikleştir**, sonra doğrula.
3. **Uzunluk** → **tür** → **aralık** → **biçim** sırasıyla.
4. **Güven sınırında**, en erken noktada.
5. **Reddet**, "temizleyip" kullanma.

İlke 1 · Beyaz liste

- **Kara liste:** "şunlar yasak" — hep bir şey unutulur.
- **Beyaz liste:** "yalnız şunlar serbest" — güvenli varsayılan.

Örnek: kullanıcı adı yalnız `[a-z0-9_]` olsun.

İlke 2 · Kanonikleştir

- Aynı şeyin farklı yazımları: `/tmp/../../etc` , `%2e%2e/` .
- Önce **tek biçime** indir (kanonikleştir), sonra denetle.
- Yoksa denetim atlatılır.

ilke 3 · `atoi` değil `strtol`

```
errno = 0;
long value = strtol(s, &end, 10);
if (end == s) return -1; /* no digits */
if (*end != '\0') return -1; /* "12abc" */
if (errno == ERANGE) return -1; /* overflowed */
if (value < min_val || value > max_val) return -1; /* business rule */
```

Neden `atoi` tehlikeli?

- `atoi("abc")` = 0 (hata sessiz)
- `atoi("999999999999")` = **tanımsız** (taşma)
- `atoi("12abc")` = 12 (fazlalık yok sayılır)

`strtol` her hatayı **size bildirir**; `atoi` bildirmez.

İlke 3 · uzunluk önekli kayıt

```
if (remaining < 3) return -1; /* no header */
uint16_t length = buffer[1] << 8 | buffer[2];
if (length > remaining - 3) return -1; /* SOURCE: declared > received */
if (length > sizeof rec->data) return -1; /* DEST: doesn't fit */
memcpy(rec->data, buffer + 3, length);
```

İki ayrı denetim, doğru sıra

- **Kaynak denetimi:** bildirilen uzunluk, gelen veriden büyük mü?
- **Hedef denetimi:** hedef tampona sığıyor mu?
- Sıra: `remaining - 3` **sarmasın** diye önce `remaining < 3`.
- Sahada: JNI'den gelen her dizi native tarafta **yeniden** doğrulanır.

CERT çifti · STR31-C

```
strcpy(copy, name);                                /* WRONG */  
int n = snprintf(copy, sizeof copy, "%s", name); /* COMPLIANT */  
if (n < 0 || (size_t)n >= sizeof copy) { /* kesildi */ }
```

`strcpy` sınır bilmez; `snprintf` boyutu bilir ve **kesilmeyi** bildirir.

CERT çifti · INT30-C

```
size_t remaining = total - read_count; /* WRONG: read_count>total -> gigantic number */  
if (read_count > total) return ERROR; /* COMPLIANT: order checked first */  
size_t remaining = total - read_count;
```

İşaretsiz çıkarma **sarar**; önce mantığı denetle.

CERT çifti · MEM30-C

```
for (n = head; n; n = n->next) free(n);           /* WRONG */  
while (n) { Node *s = n->next; free(n); n = s; }   /* COMPLIANT */
```

`free(n)` 'den sonra `n->next` okumak = bırakılmış belleğe erişim.

CERT çifti · ERR33-C

```
FILE *f = fopen(path,"rb"); fread(t,1,n,f);          /* WRONG */  
if (!f) return ERROR;  
if (fread(t,1,n,f) < n && ferror(f)) return ERROR; /* COMPLIANT */
```

Her dönüş değerini **denetle**; `fopen` NULL dönebilir.

Kural: bulguyu kimliğe bağla

Her bulguyu bir CERT kimliğiyle eşleştir:

"Bu satır MEM30-C ihlali (CWE-416)."

Böylece bulgu **nesnel** ve **aranabilir** olur.

Bölüm 1 — kısa sinama

1. Üç katman ve doğru sıra?
2. Kural ile öneri farkı?
3. `atoi` neden `strtol` 'a göre tehlikeli?

Bölüm 1 — cevaplar

1. **Sıra:** güvenli kod (kaynak) → **derleyici/araç** korumaları → **OS/çalışma zamanı** korumaları. Önce doğru yaz, sonra sertleştir, sonra çalışma anında savun.
2. **Kural** her zaman uygulanır, ihlali kusurdur; **öneri** bağlama göre iyi uygulamadır.
3. `atoi` hatayı **bildirmez** (taşma/geçersiz girdide tanımsız/0); `strtol` `endptr` + `ERANGE` ile **hatayı ve sınırı** kontrol ettirir.

2. Biçim dizesi açığı

Biçim dizesi açığı — şema

Biçim dizisi açığı

tek fark: biçim dizgesini kim belirliyor

printf(girdi)

girdi BİÇİM DİZGESİ sayılır

%x → yığını okur

%n → belleğe YAZAR

→ bilgi sızıntısı, kod akışı

printf(“%s”, girdi)

biçim dizgesi SABİT

girdi yalnız VERİ

güvenli

En tehlikelisi %n: yazdığı bayt sayısını belleğe yazar → seçili adrese yazma.

Sorun · `printf(girdi)`

```
printf(user_input);      /* WRONG */  
printf("%s", user_input); /* CORRECT */
```

İlkinde kullanıcı girdisi bir **biçim dizesi** olarak yorumlanır.

Neden tehlikeli?

`printf` biçim dizesindeki `%` belirteçlerini **komut** sayar.

Kullanıcı `%x`, `%s`, `%n` gönderirse programa iş yaptırır.

Kısa tarihçe · biçim dizesi açıkları

- **1999–2000** — **wu-ftpd** gibi yaygın FTP sunucularında gerçek zafiyetler açığı geniş çapta tanıtıldı.
- Saldırganlar günlükleme çağrılarına sızan kullanıcı verisini biçim dizesi olarak kullanıp bilgi sızdırdı, hatta uzaktan kod çalıştırdı.
- Ders: `%n` gibi "zararsız görünen" bir belirteç aslında bir **yazma astarıdır** (write primitive).
- Karşılık: derleyicide `-Wformat-security` , çalışma anında `_FORTIFY_SOURCE` .

Belirteç · `%x` , `%p`

- Yığındaki değerleri **okur** ve yazdırır.
- → **bilgi sızıntısı** (adresler, gizli değerler).
- ASLR'yi zayıflatır (adres sızarsa).

Belirteç · %s

- Yığındaki bir değeri **işaretçi** sanar ve o adresteki dizeyi okur.
- Rastgele adres → çökme ya da bellek okuma.

Belirteç · %n

- Şimdiye kadar yazılan karakter sayısını bir adrese **yazar**.
- → belleğe **yazma** → bozulma, denetim ele geçirme.
- En tehlikelisi.

Aynı hata başka yerlerde

Sadece `printf` değil:

`syslog`, `fprintf`, `snprintf`, `err` / `warn` — hepsi biçim dizesi alır.

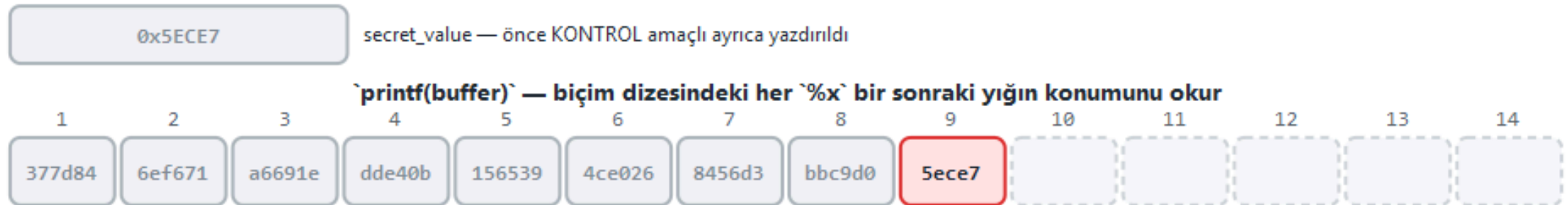
Kural: biçim dizesi **her zaman sabit** (FIO30-C).

Demo 1 — biçim dizesi

code/week-04/01-format-string · CWE-134

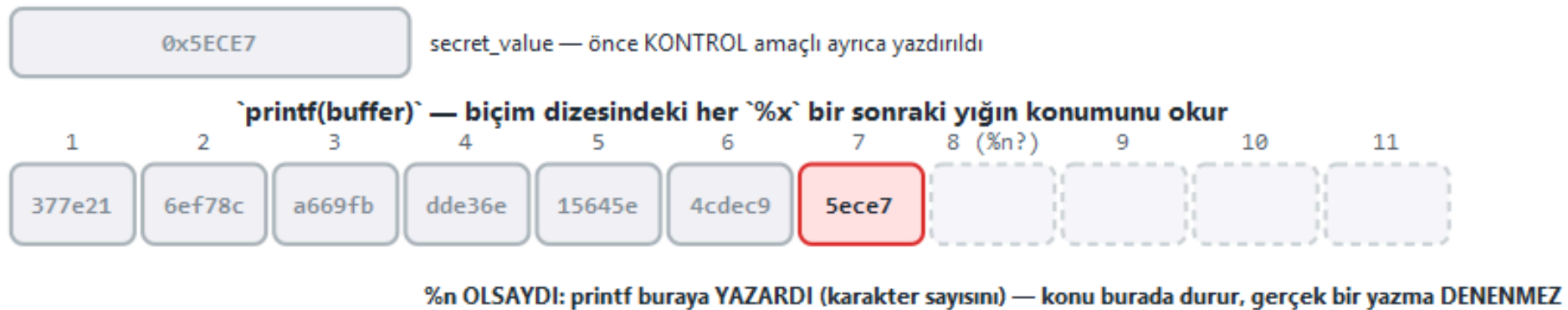
Adım	Ne görülür?
0	GCC <code>-Wformat-security</code> uyarır; MSVC <code>/analyze</code> yakalar
2 <code>%x.%x...</code>	Yığın dökülür, gizli (sentetik) değer görünür
3 <code>AAAA%n</code>	Linux korumasız çöker; Windows CRT <code>%n</code> 'i reddeder
4	<code>_FORTIFY_SOURCE : %n in writable segment</code>
5	Belirteçler yalnız metin olarak görünür

Animasyon — %x yığını tarar



Sonuç: gizli değer ekrana döküldü. Tek satırlık düzeltme (`printf("%s", ...)`) bu taramayı tamamen imkânsız kılar.

Animasyon — uç durum: `%n` bir sonraki adımda



Sonuç: gizli değer ekrana döküldü. Tek satırlık düzeltme (`printf("%s", ...)`) bu taramayı tamamen imkânsız kılar.

Düzeltilme · katman katman

1. Biçim dizesi hep **sabit** (FIO30-C).
2. Uyarı: `-Wformat -Wformat-security -Werror=format-security`.
3. Kendi fonksiyonunuzu derleyiciye **denetlettirin**:

```
void gunluk_yaz(int duzey, const char *bicim, ...)  
    __attribute__((format(printf, 2, 3)));
```

4. Çalışma anı: `_FORTIFY_SOURCE=2` ; MS CRT `%n` kapalı.

3. Bellek: use-after-free

Yığın ve öbek — şema

Yığın (stack) ve öbek (heap)

taşmanın sonucu, belleğin hangi bölgesinde olduğuna göre değişir

YIĞIN

fonksiyonla açılır/kapanır
otomatik, hızlı
boyut derleme anında belli

taşma → dönüş adresi tehlikede

ÖBEK

malloc/free ile yönetilir
yaşam süresi size bağlı
boyut çalışma anında

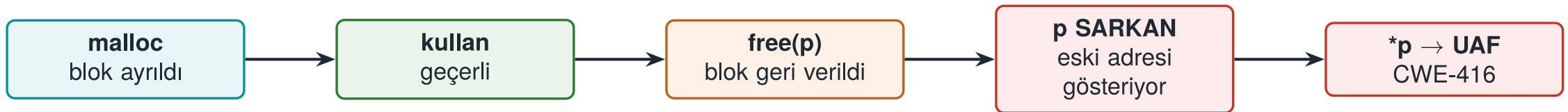
taşma → komşu blok / meta veri

Kanarya yığını korur; öbek taşmasını yakalamaz — bu yüzden ASan gerekir.

Use-after-free — şema

Use-after-free zinciri

ASan raporundaki üç iz bu zincirin üç noktasıdır



Düzeltilme free ile use ARASINDADIR: p = NULL ve tek sahiplik kuralı.

UAF nedir? · adım 1

- Bir nesne öbekte.
- **İki işaretçi** aynı nesneyi gösteriyor.

UAF · adım 2

- Biri `free` ediyor.
- Diğeri hâlâ o adresi tutuyor → **askıda (dangling)** işaretçi.

UAF · adım 3

- Ayırıcı aynı bloğu **başka nesneye** veriyor.
- Askıdaki işaretçi artık **yabancı veriyi** okur/yazar.
- Nesnede **fonksiyon işaretçisi** varsa (C++ sanal tablo) risk büyür.

UAF neden çok yaygın?

Tarayıcı/OS güncellemelerindeki "bellek bozulması düzeltildi" notlarının büyük kısmı bu sınıftır.

Karmaşık sahiplik → askıda işaretçi.

Demo 2 — UAF ve çift bırakma

code/week-04/02-use-after-free · CWE-416/415 · MEM30-C

Adım	Ne görülür?
1	Serbest blok dolar → rol <code>user</code> → <code>admin</code>
2 ASan	heap-use-after-free + yerler
3	Çift <code>free</code> → tanımsız davranış
4 ASan	attempting double-free
5	<code>free</code> + <code>NULL</code> , tek sahip → güvenli

Animasyon — küçük bir öbek

Küçük bir öbek (heap): alloc() en son serbest kalanı geri verir (LIFO)



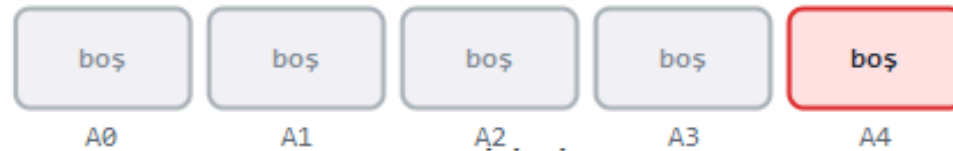
serbest liste (LIFO), üst = sıradaki

[A0, A1, A2]

Sonuç: askıdaki işaretçi üzerinden admin_panel çağrıldı — rol ve davranış saldırganın verisine döndü.

Animasyon — uç durum: çift serbest bırakma

Küçük bir öbek (heap): alloc() en son serbest kalanı geri verir (LIFO)



İKİNCİ free — aynı blok serbest listede İKİ KEZ! Liste bozuldu.

serbest liste (LIFO), üst = sıradaki

[A0, A1, A2, A3, A4, A4]

Sonuç: iki kez serbest bırakılan blok, ayırıcının bir SONRAKİ malloc çağrısında aynı belleği iki farklı sahibe vermesine yol açabilir — tanımsız davranış.

ASan raporu · üç yığın izi

```
ERROR: heap-use-after-free
#0 mode_uaf uaf.c:49  <- 1. KULLANIM
freed here:
#1 mode_uaf  uaf.c:38  <- 2. SERBEST BIRAKMA
allocated here:
#1 mode_uaf      uaf.c:32  <- 3. AYIRMA
```

Düzeltilme çoğunlukla **2. konumda**: sahiplik kararı orada yanlış.

Çözüm · C tarafı

- `free` 'den sonra işaretçiyi `NULL` yap.
- **Tek sahip**, tek bırakma noktası belirle.
- Sahiplik kuralını yaz: "bu belleği kim bırakır?"

Çözüm · C++ akıllı işaretçiler

İhtiyaç	C++
Tek sahip	<code>std::unique_ptr</code>
Paylaşılan	<code>std::shared_ptr</code>
Sahipsiz gözlem	<code>std::weak_ptr</code> → <code>lock()</code>
Dizi	<code>std::vector</code> , <code>std::array</code>

```
if (auto o = cache.lock()) use(*o); // otherwise stale memory is never accessed
```

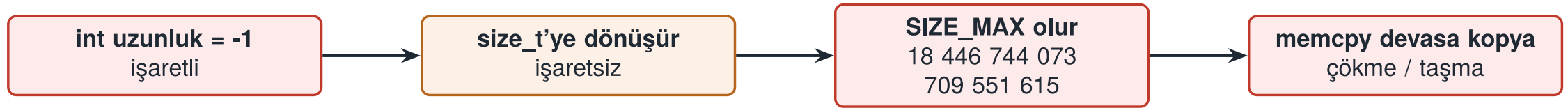
⚠ `get()` ham işaretçi; `vector` büyüyünce yineleyici geçersizleşir.

Tamsayılar ve UB

Tamsayı taşması — şema

-1 ne zaman dev olur?

işaretili → işaretsiz dönüşümü sessizce gerçekleştir



Kural: uzunlukları size_t ile tutun; dışarıdan gelen sayıyı hem alt hem üst sınırla denetleyin.

Dört tamsayı hatası

Hata	Örnek	Sonuç
İşaretsiz sarma	<code>0u - 1</code> = 4.294.967.295	Mantık hatası
İşaretli taşma	<code>INT_MAX + 1</code>	Tanımsız
Dönüşüm kaybı	<code>long long</code> → <code>int</code>	Kesilme
Kaydırma	<code>1 << 31</code>	Tanımsız

Neden önemli? · çarpım taşması

```
uint32_t size = count * 4;    /* count = 0x40000001 → size = 4 */  
array = malloc(size);        /* small block */  
/* loop writes `count` times → heap overflow */
```

Küçük ayrılan bloğa çok yazmak = taşma.

UB · derleyici denetimi silebilir

Tanımsız davranış (UB): C/C++ standardının sonucunu tanımlamadığı işlem (ör. işaretli tamsayı taşması).

```
if (x + 100 < x) /* assumes "if it overflows, it gets smaller" */  
    return -1; /* signed overflow is UB → the compiler MAY DELETE this branch */
```

- -00 'da "çalışır", -02 'de denetim **yok olur**.
- UB'ye güvenilmez.

Doğru: önce denetle

```
if ((b > 0 && a > INT_MAX - b) ||  
    (b < 0 && a < INT_MIN - b)) return false; /* before the overflow */  
if (__builtin_mul_overflow(a, b, &result)) return false; /* GCC/Clang */  
/* C23: <stdckdint.h> ckd_add/ckd_mul · MSVC: <intsafe.h> */
```

Taşmayı **olmadan önce** yakala.

Demo 3 — UB ve UBSan

code/week-04/03-undefined-behavior · CWE-190/758

- İşaretli taşma, geçersiz kaydırma, hizasız erişim
- x86'da çoğu **sessizce** yanlış sonuç
- `-fsanitize=undefined` → `signed integer overflow: 2147483647 + 1 ...`
- Windows: UBSan yok → `/RTC` , `/analyze` , CERT



`-fwrapv` mantık hatasını **gizler**; tanı için kullanma.

Animasyon — INT_MAX + add

``x = INT_MAX``; her ``add`` için ``checked_add(x, add, &y)`` mi, çıplak ``x + add`` mi?

INT_MAX = 2147483647

add = 1

y = -2147483648 (SARDI!)

add = -1

y = 2147483646

add = 100

y = -2147483549 (SARDI!)

add = -100

y = 2147483547

add = 5

y = -2147483644 (SARDI!)

add = -5

y = 2147483642

add = 1000000

y = -2146483649 (SARDI!)

add = -1000000

y = 2146483647

add = 2

y = -2147483647 (SARDI!)

add = -2

y = 2147483645

5/10 toplama tanımsız davranışa (taşma) uğradı — hepsi "başarılı" göründü, hiçbirini çökmedi. UBSan bunları çalışma anında yakalar.

Animasyon — uç durum: güvenli sürüm aynı girdiyi reddediyor

``x = INT_MAX`; her `add` için `checked_add(x, add, &y)` mi, çıplak `x + add` mi?`

INT_MAX = 2147483647

add = 1

REDDEDİLDİ — y hiç yazılmadı

add = -1

kabul edildi: y = 2147483646

add = 100

REDDEDİLDİ — y hiç yazılmadı

add = -100

kabul edildi: y = 2147483547

add = 5

REDDEDİLDİ — y hiç yazılmadı

add = -5

kabul edildi: y = 2147483642

add = 1000000

REDDEDİLDİ — y hiç yazılmadı

add = -1000000

kabul edildi: y = 2146483647

add = 2

REDDEDİLDİ — y hiç yazılmadı

add = -2

kabul edildi: y = 2147483645

5/10 toplama reddedildi; kalanı doğru sonuçla kabul edildi. Hiçbir taşma SESSİZCE geçmedi.

Hata işleme ve sinyaller

Sinyal işleyici — şema

Sinyal işleyicide ne yapılır?

sinyal, kesilen kodun ortasında gelebilir

YANLIŞ

printf, malloc, free
kilit alan her şey

yeniden girişte iç yapı bozulur
→ kilitlenme / çökme

DOĞRU

yalnız bayrak kur:
`volatile sig_atomic_t`

asıl işi ana döngü yapar
(`write()` gibi güvenli çağrılar)

Async-signal-safe olmayan fonksiyon çağırmak tanımsız davranıştır.

Denetlenmeyen dönüş değeri

Çağrı	Denetlenmezse
<code>malloc</code>	NULL'a yazma
<code>setresuid</code>	Yetkili kalır
<code>RAND_bytes</code>	"Rastgele" anahtar sıfır
<code>EVP_DecryptFinal_ex</code>	Kurcalanmış veri doğrulanmış sanılır

Hata işleme kuralları

1. Her dönüşü denetle (ERR33-C); `[[nodiscard]]`.
2. Hatada **güvenli duruma** dön (`goto temizle`) — **fail-closed**.
3. İleti ipucu vermesin: "kullanıcı yok" $\neq$ "parola yanlış".

Sahadan: opak dönüş değerleri

- Native fonksiyonlar ayrıntılı **hata kodu döndürmez**.
- Yalnız başarılı/başarısız, üstelik **opak** (9. hafta).
- Ayrıntılı kod = saldırgana "hangi denetimde takıldın" haritası.
- Tanı için ayrı, güvenli kanal → ödünleşim kaydı.

Sinyal işleyici · kural

```
static volatile sig_atomic_t stop = 0;  
static void handler(int s){ (void)s; stop = 1; } /* ONLY a flag */
```

- İşleyicide `printf`, `malloc`, `free` **yok** (SIG30-C).
- Eşzamansız-güvenli olmayan fonksiyon çağırma.
- `signal()` yerine `sigaction()`.

Gerçek olay: regreSSHion

- 2024 CVE-2024-6387: zaman aşımı **sinyal işleyicisi** güvensiz bir günlük fonksiyonu çağırıyordu.
- Sonuç: uzaktan kod çalıştırma riski.
- Ders: işleyicide yalnız bayrak set et.

Bölüm 2–3 — kısa sinama

1. `%n` neden en tehlikelisi?
2. UAF raporundaki üç iz? Düzeltme hangisinde?
3. `if (x+100 < x)` neden silinebilir?
4. Sinyal işleyicide neden `printf` yok?

Bölüm 2–3 — cevaplar

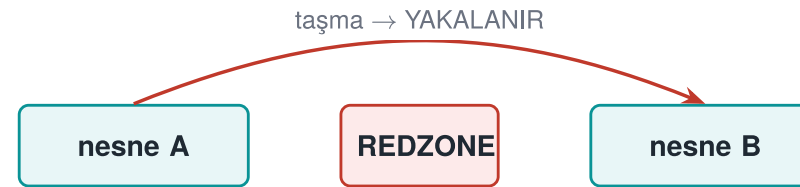
1. `%n` biçim belirtecinin yazdığı **bayt sayısını belleğe YAZAR** → saldırgan seçtiği adrese yazıp akışı ele geçirebilir.
2. **allocated-by · freed-by · use-here**. Düzeltme **freed↔use** arasında: `free` sonrası işaretçiyi `NULL` yap / yaşam süresini düzelt.
3. İmzalı taşma **tanımsız davranıştır**; derleyici "taşma olmaz" varsayar → koşul hep `false` → **siler**. Kontrolü `__builtin_add_overflow` ile yapın.
4. `printf` **async-signal-safe değil**; işleyicide yeniden girişte kilit/durum bozular → kilitlenme/çökme. Yalnız `write` gibi güvenli çağrılar.

4. Statik analiz, sanitizer, fuzzing

ASan redzone — şema

AddressSanitizer neyi yakalar, neyi kaçıır?

nesneler arasına “zehirli bölge” (redzone) koyar



alan1 | alan2
aynı struct içinde

aralarında redzone YOK
→ KAÇIRILIR

ASan ayrıca hiç çalışmayan kod yolunu da göremez — fuzzing bu yüzden gerekir.

Statik ↔ dinamik — şema

Statik analiz ↔ dinamik analiz

biri diğerinin yerine geçmez — ikisi birden kullanılır

STATİK (çalıştırmadan)

tüm kod yollarına bakar
hiç çalışmayan yolu da görür

yanlış pozitif ÇOK
bağlamı bilmez

DİNAMİK (çalıştırarak)

yalnız yürütülen yol
gerçek hatayı KESİN görür

yanlış pozitif AZ
kapsam girdiye bağlı

Fuzzing üçüncü ayakdır: dinamik analizin göreceği YENİ yolları üretir.

Statik analiz nedir?

Kodu **çalıştırmadan** inceleyip hata arama.

Avantaj: çalışmayan yolları da görür.

Dezavantaj: **yanlış alarm** üretir.

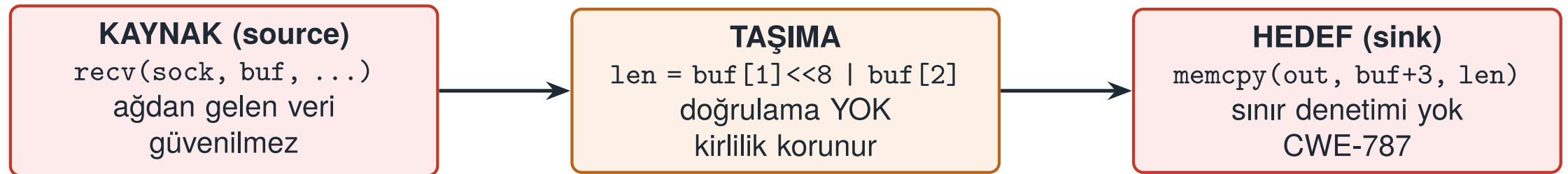
Statik analiz · katman katman

Düzyey	Araç	Bulduđu
Uyarılar	<code>-Wall -Wextra -Wconversion</code>	Dönüşüm, biçim
Analizci	<code>-fanalyzer , clang --analyze , /analyze</code>	NULL, sızıntı
Kural	<code>clang-tidy cert-* , cppcheck</code>	CERT ihlali
Anlamsal	CodeQL, Semgrep	Veri akışı
Ticari	Coverity, Klocwork	Derin analiz

Kirli veri: kaynak → hedef

Kirli veri takibi: kaynaktan hedefe

saldırganın yazdığı bir değer, denetlenmeden tehlikeli bir çağrıya ulaşıyor



Bulgu kuralı: kaynak ile hedef arasında doğrulama yoksa, o yol bir zafiyettir.

Araç, **doğrulanmamış** verinin tehlikeli bir yere aktığını görür.

Yanlış alarmla yaşamak

1. Önce **yüksek güvenilirlikli** kurallar.
2. Her bastırma **gerekçeli**: `// NOLINT(cert-err33-c): neden`.
3. Yeni kodda **sıfır uyarı** (CI zorlar).

Değerlendirici geliştiricinin kendi tarama raporunu ister.

Sanitizer nedir?

Programı **çalıştırırken** bellek/UB hatalarını yakalayan araç.

Yanlış alarm neredeyse **yok**; ama yalnız **çalışan yoldaki** hatayı bulur.

→ test + fuzzing ile birlikte.

Kısa tarihçe · AddressSanitizer

- **2012** — Google, **AddressSanitizer**'i (ASan) yayımladı: derleme anında enstrümantasyon + gölge bellek ile taşma/UAF'ı **anında** yakalıyor.
- Öncesinde bu sınıf hata genelde **sessizce** çöküyor ya da hiç fark edilmiyordu.
- ASan + fuzzing birlikte kullanılınca, çökmeyen ama belleği bozan girdi de yakalanır (bu haftanın Bölüm 4/11'i).

Sanitizer ailesi

Sanitizer	Bulduğu	Bayrak
ASan	Taşma, UAF, çift bırakma	<code>-fsanitize=address</code>
LSan	Sızıntı	ASan ile
UBSan	Taşma, kaydırma, NULL	<code>-fsanitize=undefined</code>
MSan	İlklendirilmemiş okuma	<code>-fsanitize=memory</code>
TSan	Veri yarışı	<code>-fsanitize=thread</code>

ASan nasıl çalışır? · gölge bellek

- Her 8 bayt için 1 **gölge bayt**: ne kadarı geçerli?
- Dizilerin önüne/arkasına **koruma bölgesi** (redzone).
- Serbest bloklar **karantinada** bekler.
- Her erişimden önce gölge denetimi.

ASan'ın kör noktası

```
struct { char name[8]; long role; } k;  
strcpy(k.name, long_input); /* name → role overflows */
```

Aynı yapının **içinde** alandan alana taşma → arada koruma bölgesi yok → ASan **görmez** (1. hafta Demo 3).

Sanitizer sürüme gitmez

```
gcc -g -O1 -fno-omit-frame-pointer \  
-fsanitize=address,undefined p.c -o p_test
```

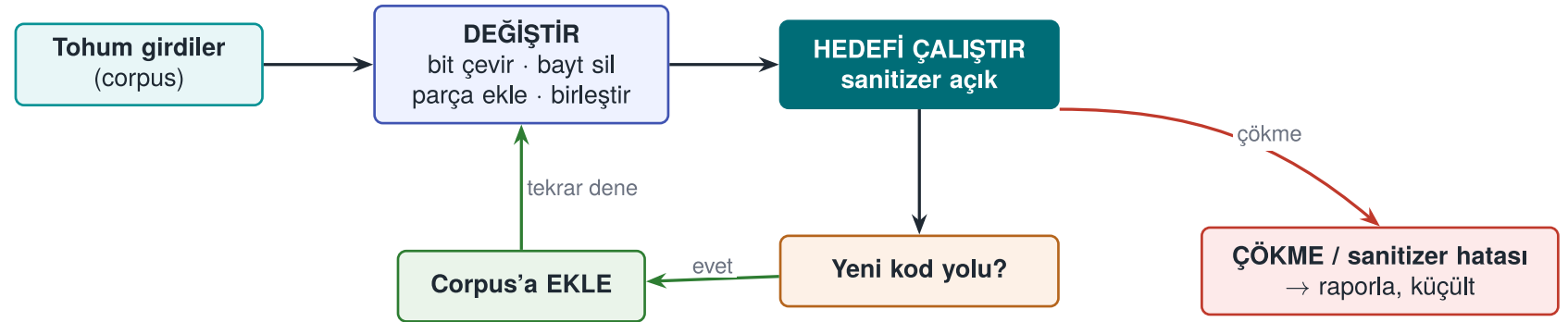
- Yavaşlatır ($\sim 2\times$), bellek kullanır.
- Yalnız **test/CI**'da; sürümde kapalı.

Fuzzing nedir? · kapsam güdümlü

Fuzzing: bir programı çok sayıda otomatik üretilmiş, çoğu bozuk girdiyle çalıştırıp çökme arama tekniği.

Kapsam güdümlü fuzzing döngüsü

rastgele değil: yeni kod yolu bulan girdiyi saklar



İyi fuzz hedefi: hızlı · deterministik · kendine yeterli · dar girdi yüzeyi.

Yeni kod yolu açan girdi **saklanır** ve üzerinde çalışılır.

Fuzzing ne bulur?

- Sihirli baytları, doğru uzunluk alanını **kendiliğinden** bulur.
- İnsanın düşünmediği girdileri.
- Araçlar: libFuzzer, AFL++, honggfuzz, OSS-Fuzz (on binlerce hata).

Kısa tarihçe · fuzzing

- **1990** — Barton Miller'ın "Fuzz" çalışması: rastgele girdiyle Unix araçlarının %25-33'ünü çökertti; fuzzing'in akademik başlangıcı.
- **2013** — **AFL** (american fuzzy lop): kapsam geri beslemesini pratik ve yaygın hâle getirdi.
- Ardından **libFuzzer** (derlemeye gömülü, hızlı) ve OSS-Fuzz (büyük açık kaynak projelerini sürekli fuzzlayan servis).

Fuzz hedefi yazmak

```
int LLVMFuzzerTestOneInput(const uint8_t *data, size_t size) {  
    parse_document((const unsigned char *)data, size);  
    return 0;  
}
```

```
clang -g -O1 -fsanitize=fuzzer,address fuzz.c parser.c -o f  
./f corpus/ -max_total_time=10
```

İyi fuzz hedefi

Dört özellik:

- **Hızlı** (ağ/disk yok)
- **Deterministik** (aynı girdi → aynı sonuç)
- **Durumsuz** (önceki çağrıdan etkilenmez)
- **Tek iş** (bir fonksiyonu sınar)

Demo 4 — fuzzing ile hata bulmak

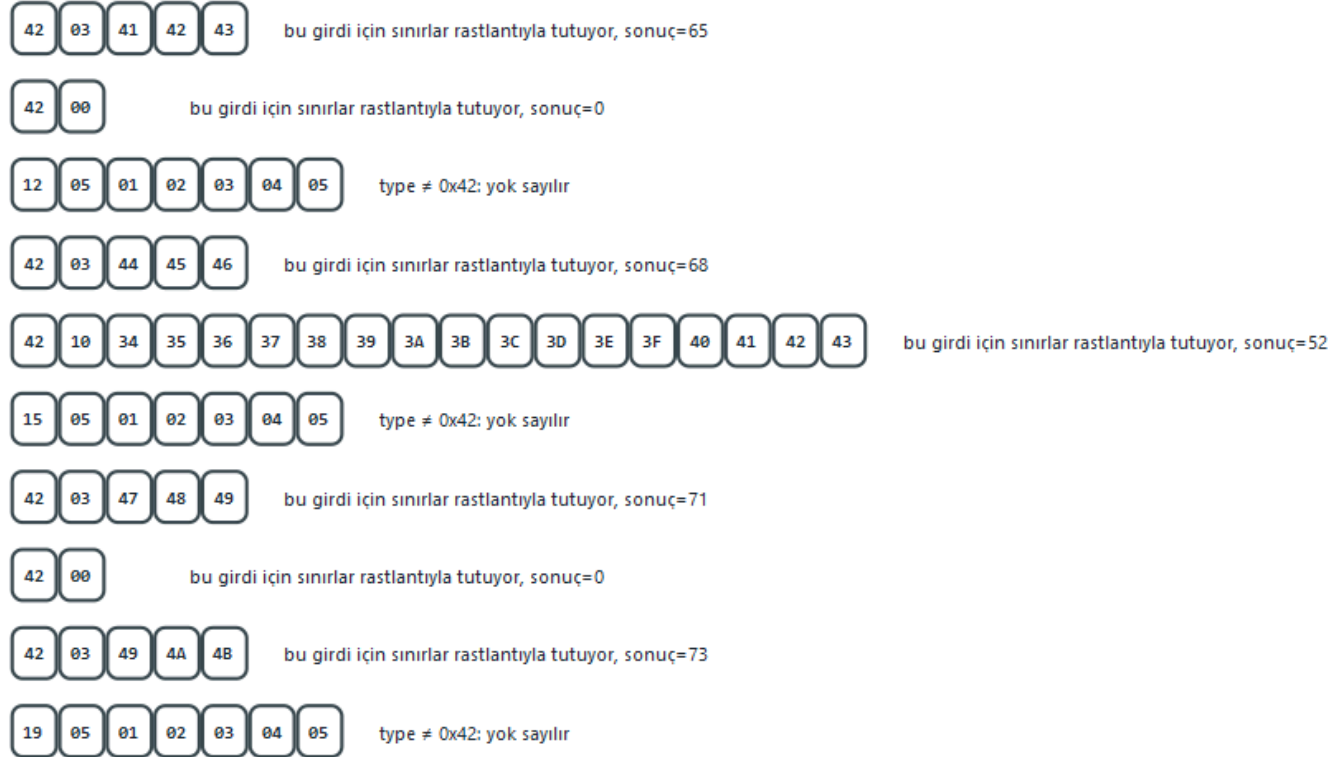
code/week-04/04-fuzzing · CWE-125/787

Adım	Ne olur?
1	seeds/normal.bin sorunsuz
2	libFuzzer ≤ 10 sn → çökerten girdi
3	ASan ile yeniden oynat → tam rapor
4	Düzeltilmiş sürüm → çökme yok
5	crash.bin güvenle reddedilir

"Şu fonksiyon şu süre fuzzlandı" = güçlü kanıt (S16).

Animasyon — izin listesi ayrıştırıcı

parser (hatalı): denetim yok — uzunluk ve veri doğrudan okunur



0/10 kayıt sınır dışı erişime yol açtı (ASan bunları "heap-buffer-overflow" olarak yakalar). Kalanları "şanslıyız" diye güvenli saymak yanlıştır.

Animasyon — uç durum: hepsi bozuk kayıt

parser (hatalı): denetim yok — uzunluk ve veri doğrudan okunur

(boş girdi) $n < 1$: güvenle 0 döner

42 $n = 1$: `data[1]` OKUNUYOR ama yok — SINIR DIŞI OKUMA (tanımsız davranış)

42 13 $\text{length} = 19$ denetlenmedi: döngü `data[2..20]`'e kadar okuyor — SINIR DIŞI (girdi ve/veya 16 baytlık hedef aşıyor)

42 0A 01 02 03 $\text{length} = 10$ denetlenmedi: döngü `data[2..11]`'e kadar okuyor — SINIR DIŞI (girdi ve/veya 16 baytlık hedef aşıyor)

(boş girdi) $n < 1$: güvenle 0 döner

42 $n = 1$: `data[1]` OKUNUYOR ama yok — SINIR DIŞI OKUMA (tanımsız davranış)

42 11 $\text{length} = 17$ denetlenmedi: döngü `data[2..18]`'e kadar okuyor — SINIR DIŞI (girdi ve/veya 16 baytlık hedef aşıyor)

42 0A 01 02 03 $\text{length} = 10$ denetlenmedi: döngü `data[2..11]`'e kadar okuyor — SINIR DIŞI (girdi ve/veya 16 baytlık hedef aşıyor)

(boş girdi) $n < 1$: güvenle 0 döner

42 $n = 1$: `data[1]` OKUNUYOR ama yok — SINIR DIŞI OKUMA (tanımsız davranış)

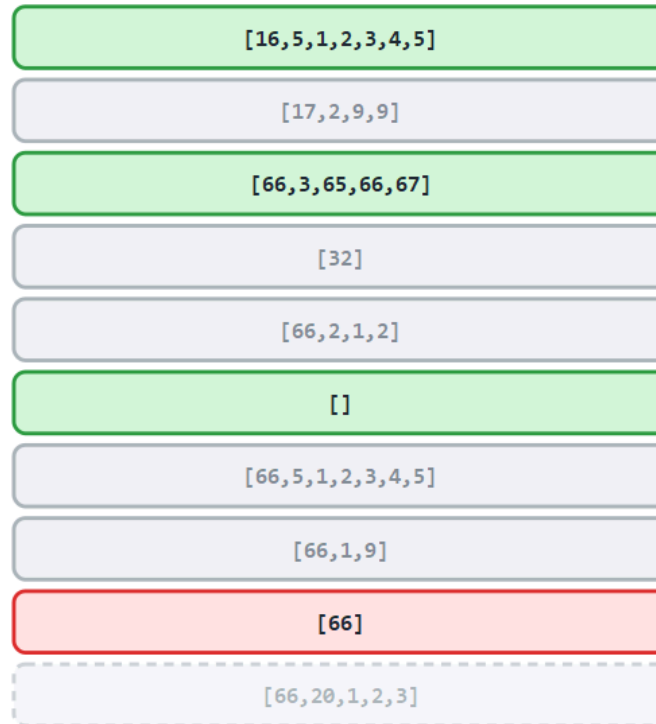
7/10 kayıt sınır dışı erişime yol açtı (ASan bunları "heap-buffer-overflow" olarak yakalar). Kalanları "şanslıyız" diye güvenli saymak yanlıştır.

Animasyon — kapsam güdümlü fuzzing

Kapsam güdümlü fuzzing: yeni bir bölge açan girdi KORUNUR, tekrar eden ATLANIR

korpus (korunan girdiler):

[#1, #3, #6, #9]



uzunluk baytı YOK ($n < 2$) — ÇÖKME

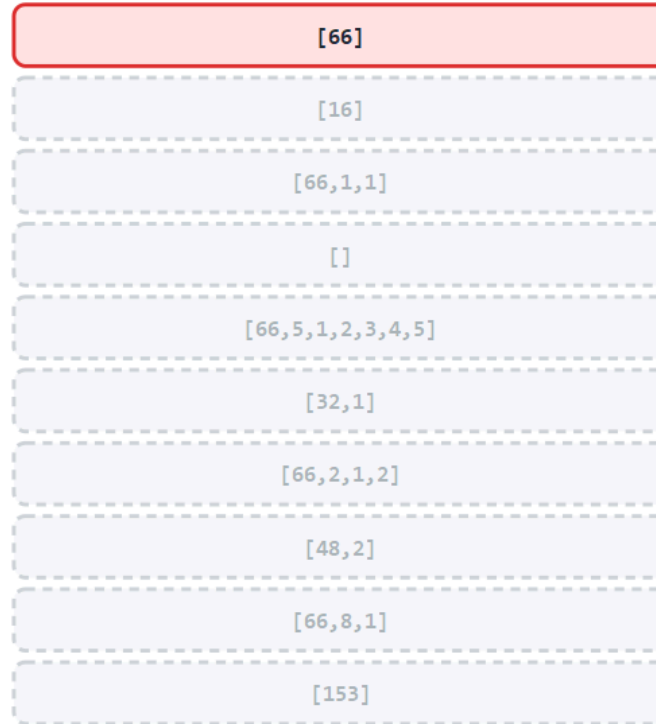
4 bölge keşfedildi, 9. adayda bir çökme bulundu (1 aday hiç denenmedi — oturum durdu).

Animasyon — uç durum: ilk adayda hemen çökme

Kapsam güdümlü fuzzing: yeni bir bölge açan girdi KORUNUR, tekrar eden ATLANIR

korpus (korunan girdiler):

[#1]



uzunluk baytı YOK ($n < 2$) — ÇÖKME

1 bölge keşfedildi, 1. adayda bir çökme bulundu (9 aday hiç denenmedi — oturum durdu).

Bölüm 4 — kısa sinama

1. Statik ve dinamik analizin farkı?
2. ASan hangi taşmayı göremez?
3. İyi fuzz hedefinin dört özelliği?

Bölüm 4 — cevaplar

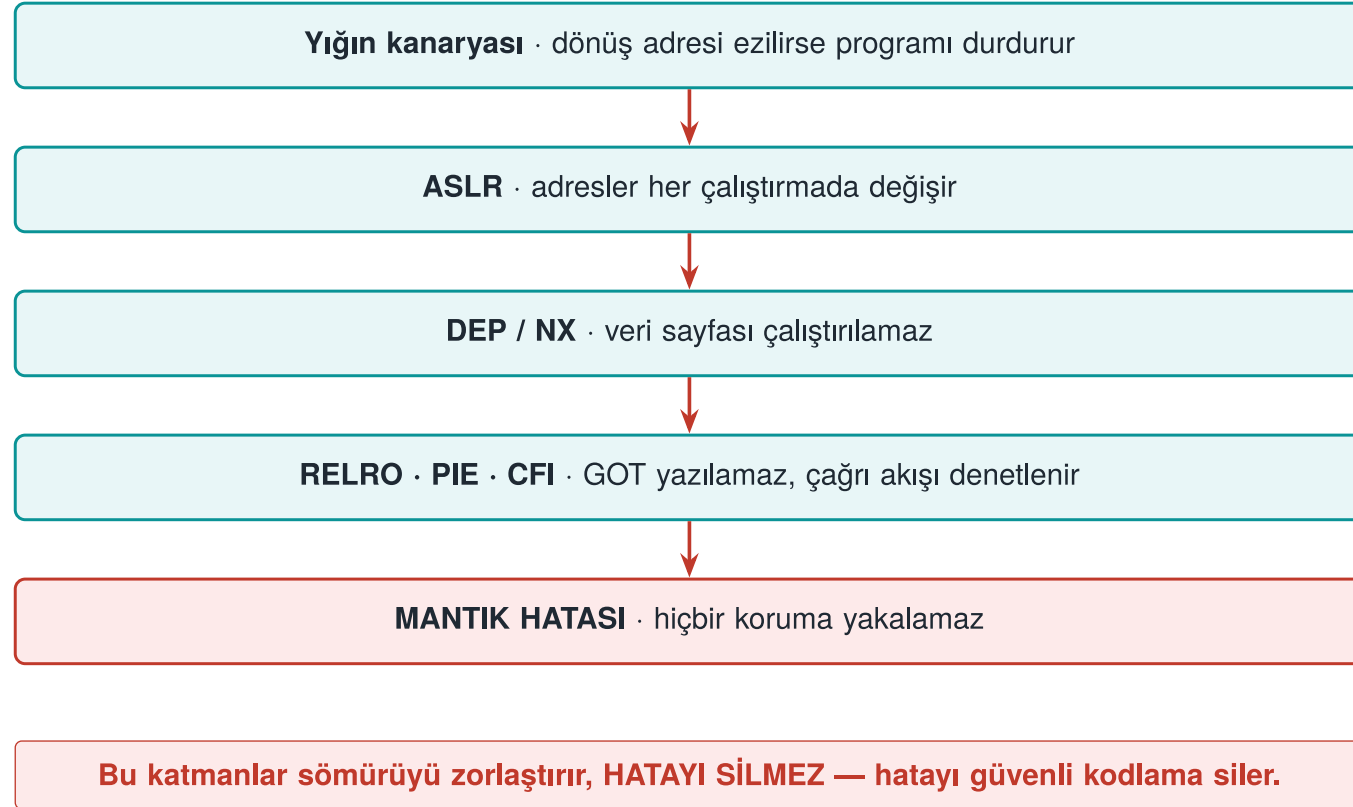
1. **Statik:** kodu çalıştırmadan tüm yolları inceler (yanlış pozitif olabilir). **Dinamik:** çalıştırırken yalnız yürütülen yolda **gerçek** hatayı gözler.
2. ASan **redzone** koyduğu yerleri görür; bir nesnenin **içindeki** alanlar arası taşmayı ve **hiç çalışmayan** yolları göremez.
3. **Hızlı, deterministik, kendine yeterli** (dış duruma bağlı değil), **dar bir girdi yüzeyi** işleyen; çökmede net başarısızlık veren hedef.

5. Derleyici ve OS korumaları

Derleyici/OS korumaları — şema

Derleyici ve işletim sistemi korumaları

saldırgan yukarıdan aşağı her katmanı ayrı ayrı aşmak zorunda



Katman 3 · fikir

Hata kaçtı diyelim.

- Sömürüyü **pahalı** kılan otomatik korumalar var.
- Çoğu bir **derleme bayrağıyla** açılır.
- Bedava değil ama ucuz.

Yığın kanaryası

- Dönüş adresinden önce bir **nöbetçi değer** konur.
- Taşma bu değeri bozarsa, dönüşten önce fark edilir ve program **durur**.
- `-fstack-protector-strong` / MSVC `/GS` .

Kısa tarihçe · derleyici/OS korumaları

- **1998** — **StackGuard**: yığın kanaryası fikrini derleyiciye ilk kez otomatik olarak ekledi.
- **2001** — **PaX** projesi, Linux çekirdeğine **ASLR**'ı (adres uzayı düzeni rastgeleleştirme) getirdi.
- **2003–2004** — **W^X / NX / DEP**: bellek sayfaları aynı anda hem yazılabilir hem çalıştırılabilir olamaz.
- Bu üçü birlikte, günümüz derleyici/OS korumalarının temelini oluşturur.

Yığın taşması ve kanarya — şema

Yığın taşması ve kanarya

taşma tampondan dönüş adresine doğru yazar



Durdurmaz: hedefli yazma · öbek (heap) taşması · bilgi sızıntısı.

FORTIFY_SOURCE

- Boyutu bilinen tampona fazla yazan **kütüphane çağrılarını** yakalar.
- `-D_FORTIFY_SOURCE=2` (+ en az `-O1`).
- Kendi döngülerinizi **korumaz**.

PIE + ASLR

- **ASLR:** adresleri rastgeleleştirir.
- **PIE:** programın da rastgele yüklenmesini sağlar.
- `-fPIE -pie / /DYNAMICBASE /HIGHENTROPYVA`.

DEP/NX ve RELRO

- **DEP/NX:** veri bölgesini çalıştırılmaz yapar (varsayılan).
- **RELRO:** GOT gibi tabloları salt-okunur yapar → `-Wl,-z,relro,-z,now` .

CFI / CFG ve gölge yığın

- **CFI/CFG:** dolaylı çağrıların yalnız **geçerli** hedeflere gitmesini denetler.
- `-fcf-protection` , `-fsanitize=cfi` / `/guard:cf` .
- **Gölge yığın:** dönüş adreslerinin ikinci kopyası (CET).

Korumalar · tek tablo

Koruma	GCC/Clang	MSVC
Kanarya	<code>-fstack-protector-strong</code>	<code>/GS</code>
FORTIFY	<code>-D_FORTIFY_SOURCE=2</code>	Güvenli CRT
PIE/ASLR	<code>-fPIE -pie</code>	<code>/DYNAMICBASE</code>
NX	varsayılan	<code>/NXCOMPAT</code>
RELRO	<code>-Wl,-z,relro,-z,now</code>	—
CFI	<code>-fcf-protection</code>	<code>/guard:cf</code>



Her koruma sınırlıdır

Koruma	Durdurur	Durdurmaz
Kanarya	Dönüş adresine taşma	Öbek taşması, sızan kanarya
FORTIFY	Kütüphane çağrısı taşması	Kendi döngüleriniz
ASLR	Sabit adres varsayımı	Adres sızıntısı
NX	Veri bölgesinde kod	Kod yeniden kullanımı (ROP)

Ana kural

Her koruma bir **tekniği** zorlaştırır, bir **hata sınıfını** kaldırmaz.

Mantık hatasına **hiçbiri** yetmez → önce güvenli kodlama.

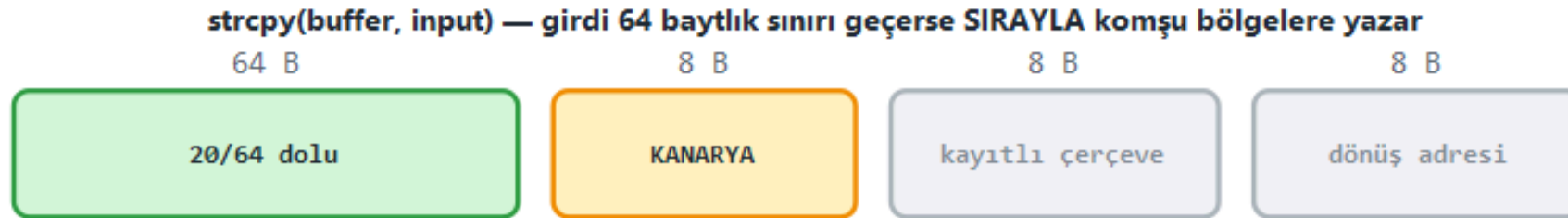
Demo 5 — korumaları açıp kapatmak

code/week-04/05-compiler-protections · CWE-121

- Aynı taşma: `overflow_weak` (kapalı) vs `overflow_hardened` (açık).
- Sert: `*** stack smashing detected ***` / Windows `0xC0000409`.
- Zayıf: sessiz bozulma ya da çökme.

```
readelf -h p | grep Type          # DYN = PIE
readelf -d p | grep BIND_NOW      # tam RELRO
readelf -s p | grep __stack_chk   # kanarya
```

Animasyon — yığın kanaryası



20 karakter tam olarak buffer'a sığıyor (taşma yok). Fonksiyon normal döner; kanarya (varsa) hiç dokunulmamış olarak kalır.

Animasyon — uç durum: aynı taşma, ZAYIF sürümde kanarya yok

strcpy(buffer, input) — girdi 64 baytlık sınırı geçerse SIRAYLA komşu bölgelere yazar

64 B

8 B

8 B

64/64 dolu

kayıtlı çerçeve

dönüş adresi

Bu yapıda denetleyecek bir kanarya YOK: fonksiyon "normal" döner ama kaydedilmiş çerçeve ve/veya dönüş adresi artık saldırganın verisi — bir SONRAKİ adımda (fonksiyon dönüşünde) çökme ya da daha kötüsü olabilir.

Önerilen sürüm bayrakları

GCC/Clang:

```
-O2 -D_FORTIFY_SOURCE=2 -fstack-protector-strong -fPIE -pie -Wl,-z,relro,-z,now -fcf-protection -Wall -Wextra -Wformat=2 -Werror=format-security
```

MSVC:

```
/O2 /GS /sdl /guard:cf /DYNAMICBASE /HIGHENTROPYVA /NXCOMPAT /CETCOMPAT /W4
```

Koruma tablosu = kanıt

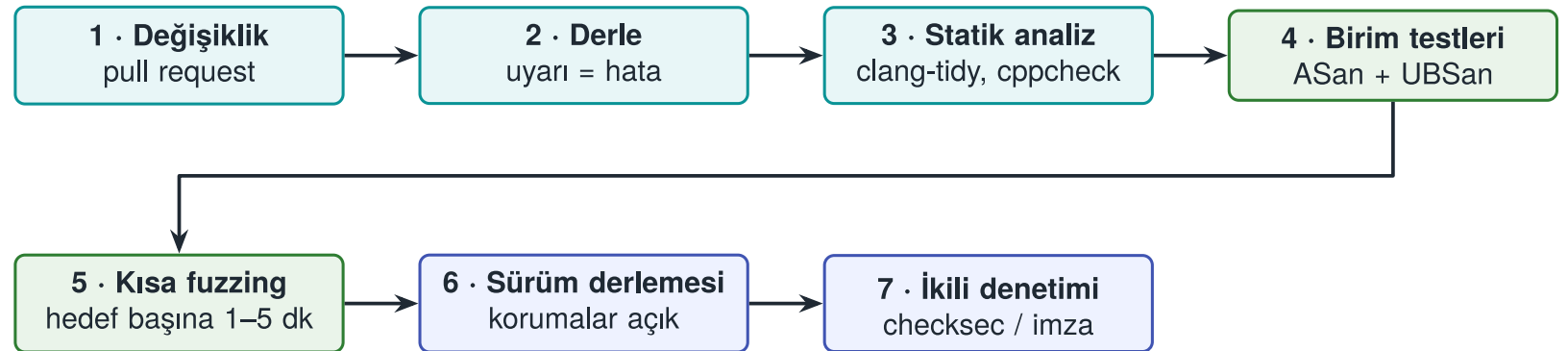
- Değerlendirici her binary ve `.so` için **koruma tablosu** çıkarır.
- Kapalı koruma bir **bulgudur**; gerekçesi ödünleşim kaydında olmalı.
- Şema: uygulama, korumaların açık olduğunu **kendisi** de denetleyebilir.

Güvenli derleme hattı (CI)

CI (Continuous Integration): her kod değişikliğinde otomatik derleme + test çalıştıran sistem.

Güvenli derleme hattı (CI)

her adım geçmeden bir sonrakine geçilmez



Ucuzdan pahalıya: erken adımlar kolay hataları eler, pahalı adım yalnız kalanlara harcanır.

Her hatalı adım birleştirmeyi **durdurur**.

CI · başarısızlık ölçütü

Aşama	Başarısızlık
Derleme	Tek uyarı
Sanitizer'lı test	Herhangi bir rapor
Fuzzing	Yeni çökme (eski corpus = regresyon)
Binary denetimi	Günlük dizesi, sembol, kapalı koruma

Dersin CMake kipleri

Kip	Bayraklar
korumasiz	-00
optimize	-02
asan	-01 -fsanitize=address
denetimli	-02 -D_FORTIFY_SOURCE=2
guvenli	-02 + düzeltilmiş kaynak

Koruma bayraklarının çoğu: Demo 5 `overflow_hardened` .

Derleme ortamı da hedeftir

- Erişim kaydı; bağımlılık **sabitleme** (5. hafta SBOM).
- İmzalı sürüm; **yeniden üretilebilir** derleme.
- Derleme makinesi ele geçerse tüm ürün etkilenir.

Bölüm 5 — kısa sinama

1. Kanarya neyi durdurur, neyi durdurmaz?
2. ASLR neden bir bilgi sızıntısıyla zayıflar?
3. Neden "hiçbir koruma mantık hatasına yetmez"?

Bölüm 5 — cevaplar

1. **Durdurur:** dönüş adresini ezen **ardışık** yığın taşmasını (kanarya bozulu). **Durdurmaz:** hedefli yazma, heap taşması, bilgi sızıntısı.
2. Tek bir **sızan adres** modülün taban adresini açığa çıkarır (offset sabit) → tüm rastgelelik çöker.
3. Korumalar **bellek** ihlali yakalar; yanlış yetkilendirme/iş kuralı **geçerli** bellek erişimidir → hiçbir koruma tetiklenmez. Doğru tasarım + inceleme gerekir.

6. Kod gizlemeye giriş

Katman 4 · neden?

- Beyaz kutu saldırgan: dizeleri okur, fonksiyonları adıyla bulur, denetimi atlar.
- Gizleme: davranış aynı, **anlaşılması zor**.

İki saldırgan modeli — şema

Uzak saldırgan mı, cihazın sahibi mi?

aynı programa iki farklı tehdit modeli

UZAK SALDIRGAN

girdi gönderir
belleği göremez
bellek güvenliği + doğrulama yeter
1–5. haftalar

CİHAZIN SAHİBİ (MATE)

ikiliyi elinde tutar
hata ayıklayıcı bağlar
gizleme + RASP gerekir
6., 9., 14. haftalar

Gizlemeye buradan geçiyoruz: bellek güvenliği ikinci saldırgana karşı yetmez.

Gizlemenin amacı

İmkânsız kılmak değil, **pahalı** kılmak (Cookbook 12.1):

- maliyet > korunan değer
- otomasyonu zorlaştır (çeşitlendirme, 14. hafta)
- diğer katmanlara **zaman** kazandır

Tek gelişmiş teknik yerine **birkaç basit teknik + veri gizleme**.

Kısa tarihçe · kod gizleme taksonomisi

- **1997** — Collberg, Thomborson ve Low, gizleme (obfuscation) tekniklerini **sistematik bir taksonomiye** ayıran çalışmalarını yayımladı: yerleşim (layout), veri (data), kontrol (control) ve önleme (preventive) dönüşümleri.
- Bu hafta gördüğümüz dize/sembol gizleme = **veri** dönüşümü; kontrol akışı düzleştirme ve opak yükleme = **kontrol** dönüşümü.
- Taksonomi hâlâ akademik ve endüstriyel gizleme literatürünün ortak dilidir.

Gizleme haritası

Aile	Neyi gizler?	Hafta
Düzen	Adlar, yapı	4
Veri	Sabit, dize, değişken	4, 9
Kontrol akışı	Algoritma yapısı	4, 9
Önleyici	Analiz aracı	9
Sanallaştırma	Makine kodu	9, 14

Sahadan: native tarafta 18 önlem

Sembol görünürlüğü kapalı · ad anlamsızlaştırma · sabit aritmetik gizleme · dize şifreleme (kullanınca çöz, **hemen sil**) · opak boolean · kendi kütüphane fonksiyonları · sahte işlem/ölü dal · düzleştirme + rastgele çıkış · sürümde günlük **yok**.

➔ Güç **birlikteliktan** ve RASP'tan.

Üç ucuz önlem

Önlem	Nasıl?
Sembolleri gizle	<code>static</code> , <code>-fvisibility=hidden</code> , <code>strip -s</code>
Günlüğü kaldır	Makroyu derlemede boşalt
Dizeleri gizle	Derlemede şifrele, kullanınca çöz, hemen sil

Günlük makrosu

```
#ifndef LOG_ENABLED
#  define LOG(...) fprintf(stderr, __VA_ARGS__)
#else
#  define LOG(...) ((void)0)  /* neither the string nor the call end up in the binary */
#endif
```

Sürümde `LOG_ENABLED` tanımlı değil → günlük **hiç derlenmez**.



Çalışma anı bayrağı yetmez

```
if (debug_flag) printf("..."); /* BAD */
```

- Dize binary dosyada **kalır**.
- Saldırgan bayrağı değiştirip günlüğü **yeniden açar**.
- Doğrusu: derleme anında **çıkar**.

Dize gizleme $\neq$ anahtar saklama

- Çözülen dize kullanım anında **bellekte açık**.
- Çözme anahtarı **programın içinde**.
- Durdurduğu: `strings` gibi **statik** taramalar.
- Çalışırken inceleyene $\rightarrow$ RASP (6). Gerçek anahtar $\rightarrow$ whitebox (11).

Demo 6 — sembol ve dize sızıntısı

code/week-04/06-symbol-strings · CWE-200/215

Adım	Ne olur?
1	<code>secret_exposed</code> ve <code>secret_hidden</code> aynı sonuç
2 <code>strings</code>	Açıkta lisans dizesi ve <code>[LOG]</code> ; kapalıda yok
3 <code>nm</code>	Açıkta <code>license_verify</code> ; kapalıda sembol yok
4	Windows: adlar PDB'de; kapalı PDB üretmez

Animasyon — dize + sembol gizleme

AÇIK sürüm: dize ikilide DÜZ metin olarak duruyor

XOR key = 0x5A

C	E	N	4	2	9	-	L	I	C	E	N	S	E	-	2	0	2	6
43	45	4E	34	32	39	2D	4C	49	43	45	4E	53	45	2D	32	30	32	36

nm ile görünen semboller:

license_verify

secret_text

Özet: 19/19 bayt hâlâ yazdırılabilir aralıkta — dizinin tamamı strings çıktısında AYNEN görünür.

Animasyon — uç durum: kötü anahtar seçimi (0x00)

KAPALI sürüm: her bayt derleme anında anahtarla XOR'lanmış

XOR key = 0x00

W	E	A	K	-	X	O	R	-	K	E	Y	-	D	E	M	O	0	1
57	45	41	4B	2D	58	4F	52	2D	4B	45	59	2D	44	45	4D	4F	30	31

nm ile görünen semboller:

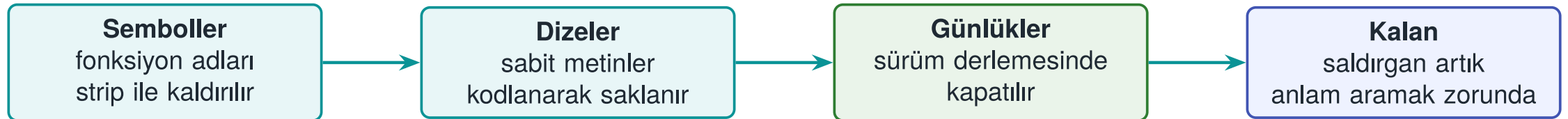
(strip: sembol yok) (strip: sembol yok)

Özet: 19/19 bayt hâlâ yazdırılabilir aralıkta — ama ardışık dizinin TAMAMI ender rastlar; strings bunu tek bir okunur sözcük olarak GÖRMEZ.

Sembol ve dize gizleme — şema

Sembol, dize ve günlük gizleme: ilk savunma hattı

tersine mühendisliğin en ucuz adımını pahalılaştırmak



En ucuz koruma budur ve çoğu projede eksiktir: `strings` çıktınızı bir kez okuyun.

Kontrol akışı düzleştirme

```
int state = 1;
for (;;) switch (state) {
    case 1: state = 2; break;
    case 2: state = (condition ? 3 : 4); break;
    case 3: state = 5; break;
    case 4: return FAILURE;
    case 5: return SUCCESS;
}
```

Blokların doğal komşuluğu kaybolur; sıra yalnız **durum değişkeninde**.

Düzleştirme: önce/sonra — şema

Kontrol akışı düzleştirme ve opak değerler

aynı davranış, okunamaz grafik

ÖNCE

if / else / while
grafik okunabilir
algoritmanın iskeleti görünür
birkaç temel blok

SONRA

tek switch + durum değişkeni
sıra değişkende saklı
opak yüklemeler sahte dal ekler
çok sayıda blok

Maliyeti vardır: boyut ve hız artar. 9. ve 14. haftada ölçmeyi öğreneceğiz.

Düzleştirmeyi güçlendirenler

- **Durum değerlerini gizle** (aritmetik dönüşüm)
- **Sahte/ölü bloklar** (hangisi gerçek?)
- **Rastgele çıkış** (hata nerede yakalandı, okunmaz)
- **Opak değerler** (tek bayt çeviremez)

Tek başına şablon kısa sürede çözülür → 9. hafta derinlik.

Demo 7 — düzleştirme

code/week-04/07-flow-flattening

Adım	Ne olur?
1	<code>pin.c</code> ve <code>pin_flattened.c</code> aynı sonuç (PIN sentetik)
2	<code>objdump</code> : düzleştirilmişte çok daha fazla dal
3	Süre: düzleştirmenin maliyeti

→ 14. hafta Tigress `Flatten` ile otomatik; 9. hafta etkinliği ölçmek.

Animasyon — dağıtıcı durum makinesi

Doğal if-zinciri: ilk başarısız denetimde hemen döner

`strlen == 4?`

`guess[0]=='4'?`

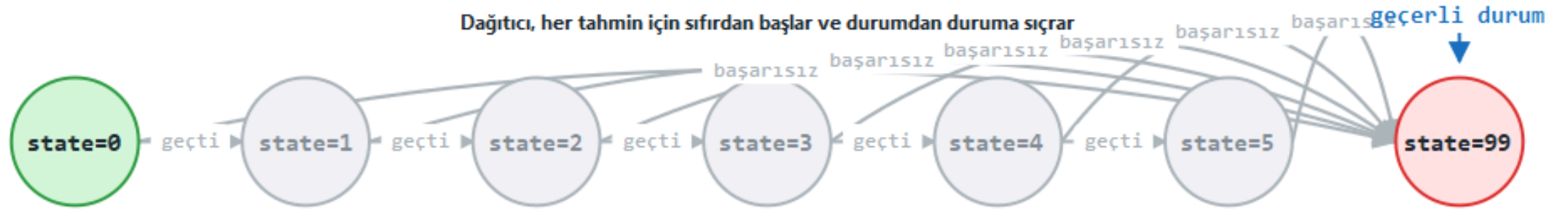
`guess[1]=='2'?`

`guess[2]=='9'?`

`guess[3]=='1'?`

2/10 tahmin doğru. Her iki sürüm de AYNI sonucu verir — yalnız kontrol akışı farklı.

Animasyon — uç durum: hepsi yanlış uzunlukta, anında ret



0/10 tahmin doğru. Her iki sürüm de AYNI sonucu verir — yalnız kontrol akışı farklı.

Animasyon — opak yüklem

	#1	#2	#3	#4	#5	#6	#7	#8	#9	#10
$x =$	0	1	2	3	4	5	6	7	8	9
$x*x =$	0	1	4	9	16	25	36	49	64	81
$(x*x)\%4 =$	0	1	0	1	0	1	0	1	0	1

``pin_verify_obfuscated("4291", x)`` — sonuç x 'e göre DEĞİŞİR Mİ?

$x=0 \rightarrow 1$

$x=9 \rightarrow 1$

10/10 değer için $(x*x)\%4 \neq 2$ doğrulandı — 2 hiç görülmedi. Yem dal (decoy_branch) tersine mühendisin okuması gereken ölü bir yoldur, hiçbir zaman çalışmaz.

Animasyon — uç durum: INT sınırlarına yakın değerler

	R1	R2	R3	R4	R5	R6	R7	R8	R9	R10
A	21670K067	-21670K06K	0K100	-0K100	0	1	-1	2	-2	00000
A*A	00110000010102020000	001100000102070000	216700000	216700000	0	1	1	0	0	0200007200
(A*A)%4	1	0	0	0	0	1	1	0	0	0

"pin_verify_obfuscated('4291', x)" — sonucu'ye göre DEĞİŞİR Mİ?

A=21670K067 -> 1

A=00000 -> 1

10/10 değer için $(x*x)\%4 \neq 2$ doğrulandı — 2 hiç görülmedi. Yem dal (decoy_branch) tersine mühendisin okuması gereken ölü bir yoldur, hiçbir zaman çalışmaz.

Kavram: 9. haftaya bağlanan üç teknik

- **Fonksiyon adı gizleme:** dışa açık adlar anlamsız; okunabilirlik makroyla.
- **Bellek ayırma gizleme:** "32 bayt = anahtar" izini bulanıklaştır.
- **Dinamik şifreleme:** hassas kod binary'de şifreli; DEP/NX ile çatışır, seçici.

Bölüm 6 — kısa sinama

1. Gizleme hatayı düzeltir mi?
2. Günlüğü çalışma anı bayrağıyla susturmak neden yetmez?
3. Düzleştirmeyi güçlendiren üç teknik?

Bölüm 6 — cevaplar

1. **Hayır.** Gizleme yalnız tersine mühendisliği zorlaştırır; açık yerinde durur. **Önce düzelt, sonra gizle.**
2. Dize/kod binary dosyada kalır; saldırgan bayrağı çevirir/dizeleri okur. Sürümde log'u **derleme zamanında** çıkarın (makro/ölü kod eleme).
3. **Opak yüklemeler + sahte bloklar/ölü dallar + durum değişkeni şifreleme & rastgele çıkış** (ayrıca ad/dize gizleme).

Uçtan uca: bir taşmayı üç katmanda kapatmak

Hatalı kod

```
void greet(const char *name) {  
    char buffer[16];  
    strcpy(buffer, name);          /* no bound */  
    printf("Hello %s\n", buffer);  
}
```

name 16 bayttan uzunsa ne olur?

Ne oluyor? · adım adım

- `buffer` yığında 16 bayt.
- `strcpy` `name` 'ı **sonuna kadar** kopyalar, sınıra bakmaz.
- 16'dan uzun `name` → komşu bellek (dönüş adresi dahil) bozulur.

Saldırı · sonuç

- Kısa taşma: komşu değişken bozulur (mantık hatası).
- Uzun taşma: **dönüş adresi** bozulur → çökme ya da denetim ele geçirme.
- Korumasızsa: ciddi açık (CWE-121).

Düzeltilme 1 · sınırlı kopyalama

```
int n = snprintf(buffer, sizeof buffer, "%s", name);
if (n < 0 || (size_t)n >= sizeof buffer) {
    /* truncated: reject or flag it */
}
```

`snprintf` boyutu bilir ve **kesilmeyi bildirir**.

Düzeltilme 2 · katmanlar

- **Kodlama:** `snprintf` (STR31-C).
- **Derleyici:** `-fstack-protector-strong` (kanarya).
- **FORTIFY:** `-D_FORTIFY_SOURCE=2` kütüphane çağrısını denetler.
- **Test:** ASan uzun girdiyle taşmayı yakalar.

Ders: tek satır, çok katman

- Bir hata: `strcpy` .
- Savunma: doğru fonksiyon + derleyici koruması + sanitizer testi.
- Ama **asıl** çözüm ilk satırda: sınırlı kopyalama.

Klasik hatalar — özet

Hata	Bölüm	Kural
Uyarıları görmezden gelmek	1	<code>-Werror</code> ile hataya çevir; CERT taraması temiz olsun
Girdiyi "temizleyip" kullanmak	1	Tehlikeli girdiyi reddet; beyaz liste + red en güvenlisi
Dönüş değerini denetlememek	2–3	Her dönüşü denetle (ERR33-C); <code>fopen / malloc</code> NULL dönebilir
<code>-fwrapv</code> ile UB'yi gizlemek	2–3	Taşmayı "tanımlı" yapmak mantık hatasını gizler; UBSan ile tanı koy
Sanitizer'ı sürüme koymak	4	Yalnız test/CI'da; sürüm derlemesinde kapalı
Gizlemeyle hatayı örtmek	6	Gizleme düzeltmez, yalnız saklar; önce düzelt, sonra gizle

Altı hatanın hepsi bu destede işlendi; burada tek bakışta toparlıyoruz.

7. Proje ve kapanış

Proje · S9 "Kod sağlamlaştırma" — 1

- [] Önerilen sürüm bayraklarıyla derle; koruma tablosunu ekle (`checksec` / `dumpbin`).
- [] `clang-tidy` / `cppcheck` ile CERT taraması; ≥ 5 bulguyu düzelt (kural kimliğiyle).

Proje · S9 — 2

- [] Testleri ASan + UBSan ile çalıştır.
- [] En az bir fuzz hedefi, ≥ 10 dk.
- [] Sürümde günlük yok; hassas dize `strings` çıktısında yok.
- [] Her dönüş değeri denetli (ERR33-C); `-Werror` ile derleniyor.

Önlem kartı: Açıklama → Uygulama → Doğrulama → Kalan risk.

Çözümlü kendini sinama

Soru 1

Üç katmandan hangisi hatayı gerçekten kaldırır?

Cevap: Yalnız **güvenli kodlama** (katman 2). Derleyici/OS korumaları istismarı zorlaştırır; gizleme okumayı yavaşlatır — ikisi de hatayı **kaldırmaz**.

Soru 2

`printf(girdi)` 'nin en hafif sonucu nedir?

Cevap: Bilgi sızıntısı (`%x` / `%p` ile yığın okuma). En ağırı `%n` ile belleğe yazma.

Soru 3

UAF raporundaki üç yığın izi? Düzeltme hangisinde?

Cevap: Kullanım, serbest bırakma, ayırma. Düzeltme çoğunlukla **serbest bırakma** noktasında (sahiplik kararı).

Soru 4

`if (x + 100 < x)` neden silinebilir?

Cevap: İşaretli tamsayı taşması **tanımsız davranıştır**; derleyici "taşma olmaz" varsayıp denetimi optimize edip **silebilir**.

Soru 5

Sinyal işleyicide neden `printf` yok?

Cevap: `printf` / `malloc` eşzamansız-güvenli değildir; işleyici bunları kesintiye uğramış bir durumda çağırırsa bozulma olur. Yalnız bir bayrak set edilir (SIG30-C).

Soru 6

ASan hangi taşmayı göremez?

Cevap: Aynı yapının **içinde** alandan alana taşmayı (aralarında koruma bölgesi yoktur).

Soru 7

İyi bir fuzz hedefinin dört özelliği?

Cevap: Hızlı, deterministik, durumsuz, tek iş.

Soru 8

Kanarya neyi durdurur, neyi durdurmaz?

Cevap: Durdurur: dönüş adresine ulaşan yığın taşması. Durdurmaz: öbek taşması, önceki değişkenlerin bozulması, sızan kanarya.

Soru 9

ASLR neden bir bilgi sızıntısıyla zayıflar?

Cevap: Bir adres sızarsa, saldırgan diğer adresleri ona göre **hesaplayabilir**; rastgelelik avantajı kaybolur.

Soru 10

Günlüğü çalışma anı bayrağıyla susturmak neden yetmez?

Cevap: Dize binary dosyada **kalır** ve saldırgan bayrağı değiştirip günlüğü yeniden açabilir. Derleme anında çıkarılmalı.

Soru 11

Düzleştirmeyi güçlendiren üç teknik?

Cevap: Durum değerlerini gizleme, sahte/ölü bloklar, rastgele çıkış noktası (ve opak değerler).

Soru 12

Dize gizleme neden anahtar saklama değildir?

Cevap: Çözülen dize kullanımda bellekte açıktır ve çözme anahtarı programın içindedir; yalnız statik taramayı durdurur.

Özet: bu haftanın tek cümlesi

Önce **hatasız kod** (CERT + sanitizer + fuzzing), sonra **zararı sınırla** (derleyici/OS korumaları), en son **okumayı zorlaştır** (gizleme) — ve hiçbir koruma güvenli kodlamanın yerini tutmaz.

Güvenli kod bir kez yazılmaz; **her derlemede** araçlarla denetlenir.

Sözlük

Terim	Anlam
UB	Tanımsız davranış
UAF	Serbest bırakıldıktan sonra kullanma (use-after-free)
CERT/CWE	Kural kitabı / zayıflık kataloğu
Sanitizer	Çalışma anı hata dedektörü
Fuzzing	Rastgele girdiyle çökme arama
Kanarya/ASLR/NX	Derleyici/OS korumaları

Bir sonraki hafta

5. hafta — Java ve yorumlanan diller

Bu hafta C/C++'ta elle yönetilen bellek ile derleyici/OS korumalarıyla uğraştık; 5. hafta JVM'in bu hataların (taşma, UAF, UB) çoğunu nasıl ortadan kaldırdığını, ama enjeksiyon (SQL, komut, yol) gibi yeni bir hata sınıfını nasıl açtığını gösterir.

SEI CERT Java kuralları bugünün SEI CERT C/C++ kurallarıyla karşılaştırılacak; ayrıca ProGuard/R8 ile gizleme, bağımlılık güvenliği ve SBOM işlenecek.

Hazırlık: JDK 17+, Maven · `code/week-05` .

Önce doğru kod, sonra koruma, en son gizleme. Sıra önemlidir.